

Data Visualization Project Documentation

Python Crash Course Chapters 15–17

CSC151OA1

Madison LLorente.

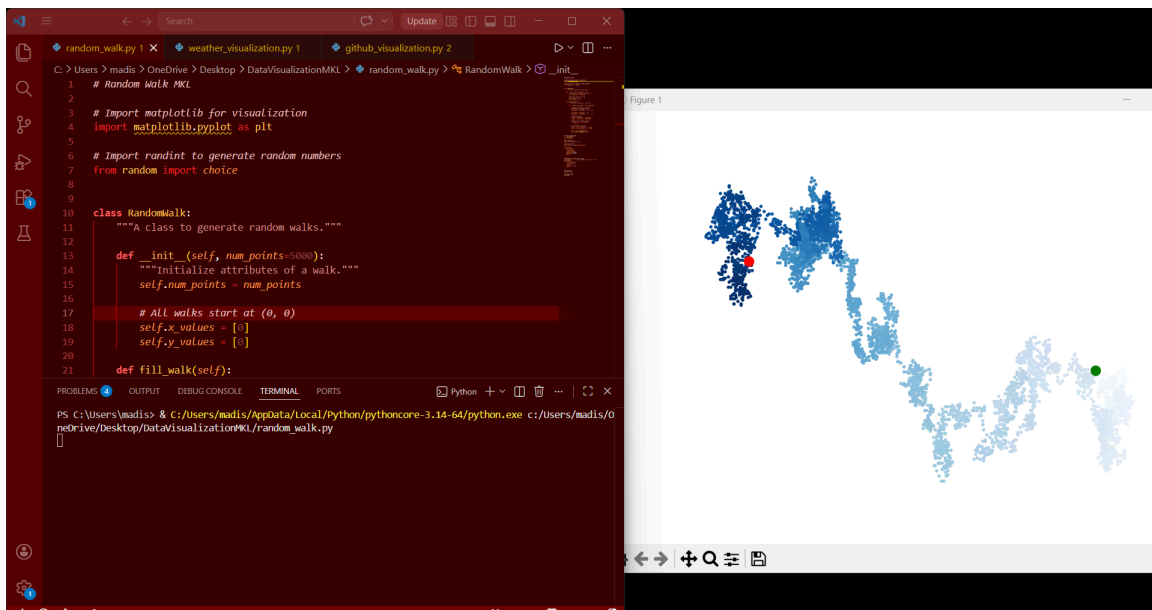
May 6, 2026

VIDEO:

<https://drive.google.com/file/d/144f1n4gzOwUZ14uPP6Nc55xntgkvmHWy/view?usp=sharing>

Program 1: Random Walk Visualization

Random_Walk.py generates a walking path by using randomly generated blue points. The program uses Python's random module and matplotlib library to display the walk. The green dot represents the walker's starting point, and the red dot its endpoint.



```
# Random Walk MKL

# Import matplotlib for visualization
import matplotlib.pyplot as plt

# Import randint to generate random numbers
from random import choice
```

```
class RandomWalk:

    """A class to generate random walks."""

    def __init__(self, num_points=5000):

        """Initialize attributes of a walk."""

        self.num_points = num_points

        # All walks start at (0, 0)

        self.x_values = [0]

        self.y_values = [0]

    def fill_walk(self):

        """Calculate all points in the walk."""

        while len(self.x_values) < self.num_points:

            # Decide direction and distance

            x_direction = choice([1, -1])

            x_distance = choice([0, 1, 2, 3, 4])

            y_direction = choice([1, -1])

            y_distance = choice([0, 1, 2, 3, 4])

            # Calculate step
```

```
        x_step = x_direction * x_distance
        y_step = y_direction * y_distance

        # Reject moves that go nowhere
        if x_step == 0 and y_step == 0:
            continue

        # Calculate new position
        next_x = self.x_values[-1] + x_step
        next_y = self.y_values[-1] + y_step

        self.x_values.append(next_x)
        self.y_values.append(next_y)

# Create random walk
rw = RandomWalk()
rw.fill_walk()

# Set chart size
plt.figure(figsize=(10, 6))

# Plot all points
point_numbers = range(rw.num_points)
```

```
plt.scatter(

    rw.x_values,

    rw.y_values,

    c=point_numbers,

    cmap=plt.cm.Blues,

    edgecolors='none',

    s=10

)

# Emphasize first and last points

plt.scatter(0, 0, c='green', edgecolors='none', s=100)

plt.scatter(

    rw.x_values[-1],

    rw.y_values[-1],

    c='red',

    edgecolors='none',

    s=100

)


# Remove axes

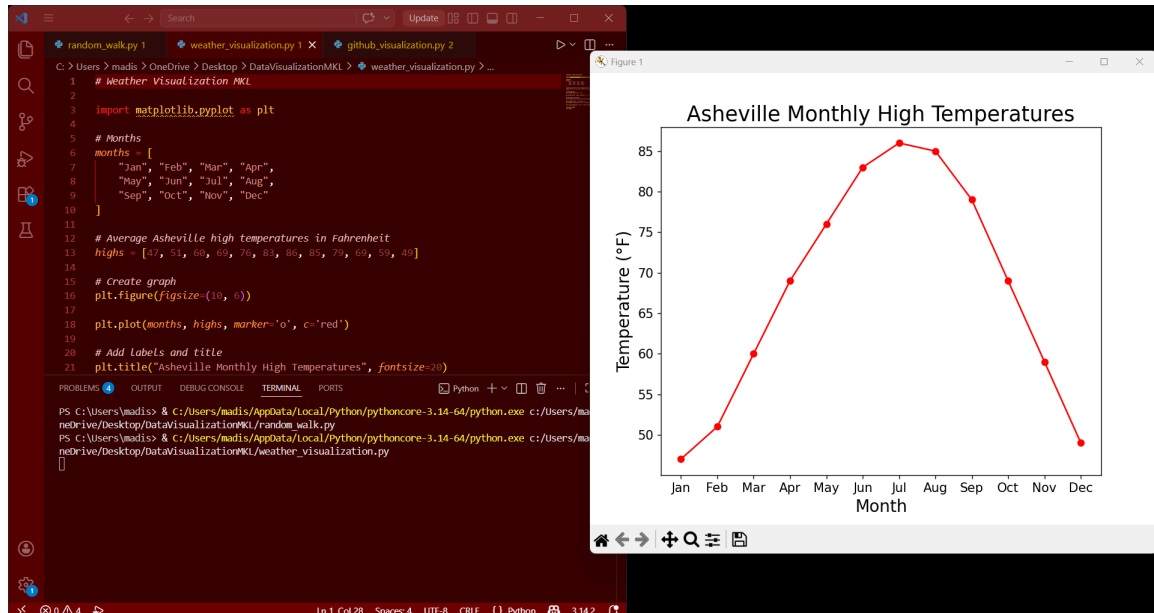
plt.axis('off')


# Display graph

plt.show()
```

Program 2: Weather Data Visualization

Weather_Visualization.py graphs average monthly high temperatures for Asheville using a climate data CSV from NOAA. The graph uses matplotlib to create a line graph displaying temperature trends throughout the year.



```
# Weather Visualization MKL

import matplotlib.pyplot as plt

# Months
months = [

    "Jan", "Feb", "Mar", "Apr",

    "May", "Jun", "Jul", "Aug",

    "Sep", "Oct", "Nov", "Dec"

]

# Average Asheville high temperatures in Fahrenheit
```

```
highs = [47, 51, 60, 69, 76, 83, 86, 85, 79, 69, 59, 49]

# Create graph

plt.figure(figsize=(10, 6))

plt.plot(months, highs, marker='o', c='red')

# Add labels and title

plt.title("Asheville Monthly High Temperatures", fontsize=20)

plt.xlabel("Month", fontsize=16)

plt.ylabel("Temperature (°F)", fontsize=16)

plt.tick_params(axis='both', which='major', labelsize=12)

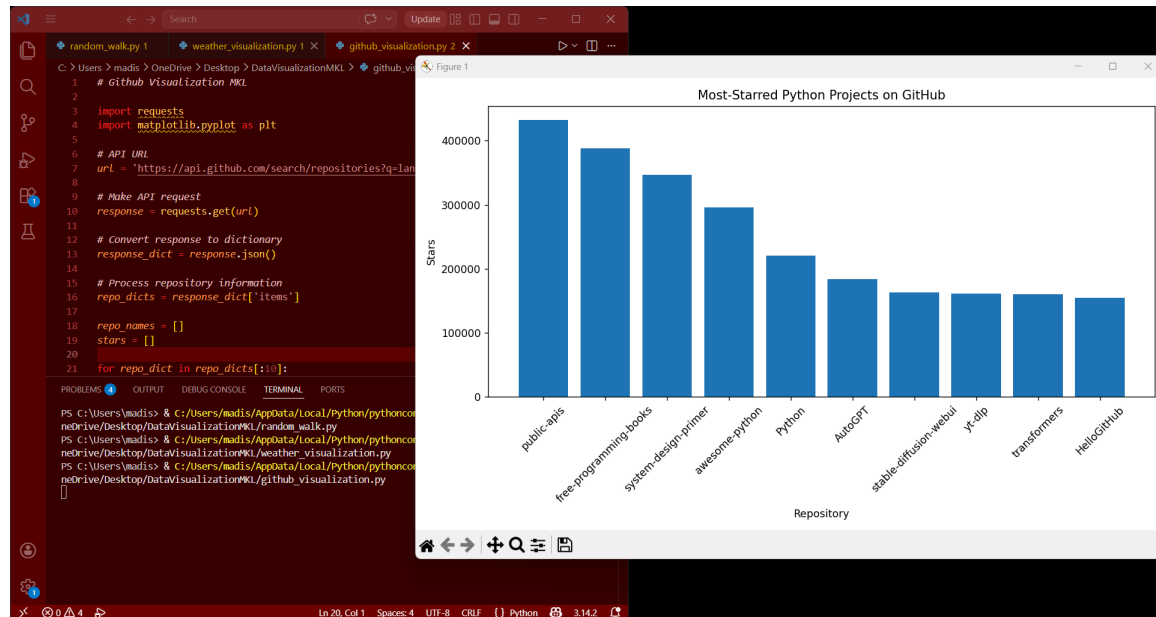
# Display graph

plt.show()
```

Program 3: GitHub API Visualization

The GitHub visualization program connects to the GitHub API and retrieves information about popular Python repositories. The program then creates a bar graph displaying

repository popularity based on star counts.



```
# Github Visualization MKL

import requests

import matplotlib.pyplot as plt

# API URL

url =

'https://api.github.com/search/repositories?q=language:python&sort=s
tars'

# Make API request

response = requests.get(url)

# Convert response to dictionary

response_dict = response.json()
```



```
# Process repository information

repo_dicts = response_dict['items']

repo_names = []

stars = []

for repo_dict in repo_dicts[:10]:

    repo_names.append(repo_dict['name'])

    stars.append(repo_dict['stargazers_count'])

# Create bar chart

plt.figure(figsize=(10, 6))

plt.bar(repo_names, stars)

# Add labels and title

plt.title("Most-Starred Python Projects on GitHub")

plt.xlabel("Repository")

plt.ylabel("Stars")

# Rotate labels

plt.xticks(rotation=45)

# Display chart

plt.tight_layout()
```

```
plt.show()
```

Installation Instructions

To run the programs successfully, install the required libraries using:

```
python -m pip install matplotlib requests
```

The programs can then be executed using:

```
python random_walk.py
```

```
python weather_visualization.py
```

```
python github_visualization.py
```

References

Matthes, E. (2016). Python crash course: A hands-on, project-based introduction to programming. No Starch Press.

Matplotlib Documentation. <https://matplotlib.org/stable/index.html>

Python Documentation. <https://docs.python.org/3/>

GitHub REST API Documentation. <https://docs.github.com/en/rest>

Stack Overflow. (2026). Stack Overflow. <https://stackoverflow.com/>